NETFLIX TITLES DATA ANALYSIS

Exploratory Data Analysis of Movies and TV Shows

PROJECT REPORT

Prepared by: Niresh

Platform: Google Colab | Language: Python

Tools: Pandas, Matplotlib, Seaborn

Project Overview

This project explores the Netflix Movies and TV Shows dataset. It removes duplicate rows, prepares missing text values, derives year-related fields, answers five descriptive questions, and creates six visualizations to summarize content types, country listings, ratings, categories, release years, and years added.

Dataset Source

Kaggle — Netflix Movies and TV Shows dataset: https://www.kaggle.com/datasets/shivamb/netflix-shows. Expected CSV filename: **netflix_titles.csv**.

Important Results Note

The original notebook has no saved execution outputs, and the CSV was not attached here. To avoid inventing findings, this report gives a results template. Run the code with the dataset and fill in the table using the actual printed values.

1. Abstract

This project performs exploratory data analysis (EDA) on a Netflix titles dataset. Its goal is to describe the catalogue rather than train a predictive model. Python and Pandas are used for loading, cleaning, and summarizing data; Matplotlib and Seaborn are used to visualize patterns in title types, countries, ratings, categories, release years, and dates added.

2. Problem Statement

Large catalogue datasets are difficult to interpret row by row. This project organizes and summarizes Netflix title information using data cleaning, descriptive counts, and charts so that major patterns can be reviewed more clearly.

3. Objectives

• Load and inspect the CSV file.
• Identify missing values and duplicate rows.
• Clean selected fields and derive year_added.
• Compare Movies and TV Shows.
• Identify common country listings, ratings, and categories.
• Explore release-year and date-added patterns.
• Produce six visualizations.

4. Dataset Description

The notebook expects **netflix_titles.csv** and uses fields such as show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in, and description. The script checks that the expected columns exist before analysis.

Field	Purpose
type	Distinguishes Movies and TV Shows
country	Country frequency analysis
rating	Content rating frequency
listed_in	Genre/category frequency
release_year	Distribution of release years
date_added	Converted to datetime to derive year_added
duration	Numeric portion extracted for exploratory correlation

5. Tools and Technologies

Python; Google Colab; Pandas for data handling; Matplotlib for plots; Seaborn for the correlation heatmap. This is a descriptive analysis project, not an ML prediction project.

6. Data Cleaning and Preparation

The code prints the dataset shape, first rows, data types, missing-value counts, and duplicate count. It removes duplicate rows, converts date_added to datetime (invalid dates become missing), fills missing values in selected text columns with Unknown, derives year_added, and extracts the numeric portion of duration into duration_number. Note that movie duration is generally in minutes and TV Show duration in seasons, so those values are not directly comparable.

7. Analysis Questions

1. How many Movies and TV Shows are present?
2. Which countries appear most frequently in country listings?
3. Which content ratings are most common?

4. Which categories or genres appear most frequently?
5. How many titles were added in each year, and which year has the highest count?

8. Visualizations

The notebook produces six charts: (1) bar chart of Movies vs TV Shows, (2) line chart of titles added by year, (3) histogram of release years, (4) scatter plot of release year vs year added, (5) pie chart of content-type share, and (6) correlation heatmap for release_year, year_added, and duration_number.

9. Results Table — Complete After Running

Use the printed notebook outputs to fill this table. Country and category counts can overlap because one title may list multiple countries or categories.

Measure	Actual result
Rows after cleaning	_____
Movies count	_____
TV Shows count	_____
Most frequent listed country	_____
Most common known rating	_____
Most frequent category/genre	_____
Year with most titles added	_____

10. Interpretation and Limitations

The results describe the particular dataset snapshot and do not establish why Netflix made a content decision. A title can list several countries and genres, so exploded frequency counts may overlap. Correlation does not prove causation. The data may contain missing or historical information and may not reflect the current catalogue.

11. Conclusion

The project demonstrates a beginner-friendly EDA workflow: inspecting data, cleaning duplicates and missing text, creating derived columns, counting categories, and visualizing patterns. To finish the submission, add actual results from the executed notebook and discuss one observed finding and one limitation.

12. Future Scope

Possible extensions include an interactive Streamlit or Power BI dashboard, filters by year/country/type, automated summary export, and comparisons across dated dataset snapshots.

13. How to Run in Google Colab

1. Download the CSV from the Kaggle link above.
2. Ensure the filename is **netflix_titles.csv**.
3. Create a Colab notebook named Project1_Netflix_Data_Analysis.
4. Upload the CSV when prompted or upload the Python source file into a runtime with the CSV available.
5. Run all code from top to bottom.
6. Fill the results table with actual outputs and save the notebook.

14. Viva Questions and Answers

Q: What is EDA? A: Inspecting, cleaning, summarizing, and visualizing data.

Q: Why remove duplicates? A: To avoid inflated counts.

Q: Why replace missing text with Unknown? A: To make missing text explicit in selected summaries.

Q: What does explode() do? A: It separates comma-delimited values into rows for individual counting.

Q: Is this a prediction model? A: No, it is descriptive analysis.

Q: Does correlation prove causation? A: No.

Appendix A — Complete Python Source Code

A separate .py file is also included for easier use.

```
"""
PROJECT 1: NETFLIX TITLES – EXPLORATORY DATA ANALYSIS (EDA)
Student: Niresh
Platform: Google Colab / Python

Dataset:
Kaggle - Netflix Movies and TV Shows
https://www.kaggle.com/datasets/shivamb/netflix-shows

Expected CSV filename: netflix_titles.csv
This is a descriptive data analysis project, not a prediction model.
"""

import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ------------------------------------------------------------
# 1. LOAD DATASET
# ------------------------------------------------------------
DATA_FILE = "netflix_titles.csv"

if not os.path.exists(DATA_FILE):
    # Helpful for Google Colab: ask user to upload the CSV if missing.
    try:
        from google.colab import files
        print("Please select the netflix_titles.csv file downloaded from Kaggle.")
        uploaded = files.upload()
        if DATA_FILE not in uploaded:
            raise FileNotFoundError(
                f"Expected '{DATA_FILE}'. Rename the downloaded dataset to "
                f"'{DATA_FILE}' and run again."
            )
    except ImportError:
        raise FileNotFoundError(
            "Place netflix_titles.csv in the same folder as this script."
        )

df = pd.read_csv(DATA_FILE)

required_cols = {
    "show_id", "type", "title", "director", "cast", "country",
    "date_added", "release_year", "rating", "duration",
    "listed_in", "description"
}
```

Appendix A — Source Code (continued)

```
missing_cols = required_cols.difference(df.columns)
if missing_cols:
    raise ValueError(
        "Dataset is missing expected columns: " + ", ".join(sorted(missing_cols))
    )

print("=" * 65)
print("NETFLIX TITLES — EXPLORATORY DATA ANALYSIS")
print("=" * 65)
print("Rows and columns before cleaning:", df.shape)
print("\nFirst five rows:")
print(df.head().to_string(index=False))
print("\nData types and non-null counts:")
df.info()
print("\nMissing values before cleaning:")
print(df.isnull().sum().to_string())
print("\nDuplicate rows before cleaning:", df.duplicated().sum())

# ------------------------------------------------------------
# 2. CLEAN AND PREPARE DATA
# ------------------------------------------------------------
df = df.drop_duplicates().copy()

# Invalid or unrecognized dates become NaT; they are not guessed.
df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")

text_cols = [
    "director", "cast", "country", "rating",
    "duration", "listed_in", "description"
]
for col in text_cols:
    df[col] = df[col].fillna("Unknown").astype(str).str.strip()
    df.loc[df[col].eq(""), col] = "Unknown"

df["year_added"] = df["date_added"].dt.year
df["duration_number"] = pd.to_numeric(
    df["duration"].str.extract(r"(\d+)")[0], errors="coerce"
)

print("\n" + "=" * 65)
print("DATA QUALITY AFTER CLEANING")
print("=" * 65)
print("Shape after cleaning:", df.shape)
print("Duplicates remaining:", df.duplicated().sum())
print("\nMissing values after cleaning:")
print(df.isnull().sum().to_string())
```

Appendix A — Source Code (continued)

```
# ------------------------------------------------------------
# 3. ANSWER FIVE DATA QUESTIONS
# ------------------------------------------------------------
print("\nQ1 — Movies vs TV Shows")
type_counts = df["type"].value_counts()
print(type_counts.to_string())

print("\nQ2 — Top countries (titles can list more than one country)")
countries = df.loc[df["country"].ne("Unknown"), "country"].str.split(", ").explode()
country_counts = countries.value_counts().head(10)
print(country_counts.to_string() if not country_counts.empty else "No country data available.")

print("\nQ3 — Top content ratings")
rating_counts = df.loc[df["rating"].ne("Unknown"), "rating"].value_counts().head(10)
print(rating_counts.to_string() if not rating_counts.empty else "No rating data available.")

print("\nQ4 — Top categories / genres (a title can have multiple categories)")
genres = df.loc[df["listed_in"].ne("Unknown"), "listed_in"].str.split(", ").explode()
genre_counts = genres.value_counts().head(10)
print(genre_counts.to_string() if not genre_counts.empty else "No category data available.")

print("\nQ5 — Titles added by year")
year_counts = df["year_added"].dropna().astype(int).value_counts().sort_index()
print(year_counts.to_string() if not year_counts.empty else "No valid date-added values available.")

if not type_counts.empty:
    print("\nLargest content type count:", type_counts.idxmax(), int(type_counts.max()))
if not country_counts.empty:
    print("Most frequently listed country:", country_counts.index[0], int(country_counts.iloc[0]))
if not rating_counts.empty:
    print("Most common known rating:", rating_counts.index[0], int(rating_counts.iloc[0]))
if not genre_counts.empty:
    print("Most frequent category:", genre_counts.index[0], int(genre_counts.iloc[0]))
if not year_counts.empty:
    print("Year with the most titles added:", int(year_counts.idxmax()), int(year_counts.max()))

# ------------------------------------------------------------
# 4. SIX VISUALIZATIONS
# ------------------------------------------------------------
sns.set_theme()  # Use Seaborn's default theme for readable plots.

# 1. Bar chart: Movies vs TV Shows
plt.figure(figsize=(7, 4))
type_counts.plot(kind="bar", title="Movies vs TV Shows")
plt.xlabel("Content type")
```

Appendix A — Source Code (continued)

```
plt.ylabel("Number of titles")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 2. Line chart: titles added by year
if not year_counts.empty:
    plt.figure(figsize=(8, 4))
    year_counts.plot(kind="line", marker="o", title="Titles Added to Netflix by Year")
    plt.xlabel("Year added")
    plt.ylabel("Number of titles")
    plt.tight_layout()
    plt.show()

# 3. Histogram: release-year distribution
plt.figure(figsize=(8, 4))
df["release_year"].dropna().plot(
    kind="hist", bins=20, title="Distribution of Release Years"
)
plt.xlabel("Release year")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 4. Scatter plot: release year vs year added
tmp = df.dropna(subset=["release_year", "year_added"])
plt.figure(figsize=(7, 4))
plt.scatter(tmp["release_year"], tmp["year_added"], alpha=0.35)
plt.title("Release Year vs Year Added")
plt.xlabel("Release year")
plt.ylabel("Year added")
plt.tight_layout()
plt.show()

# 5. Pie chart: Movies vs TV Shows share
if not type_counts.empty:
    type_counts.plot(
        kind="pie", autopct="%1.1f%%",
        title="Share of Movies and TV Shows", figsize=(6, 6)
    )
    plt.ylabel("")
    plt.tight_layout()
    plt.show()

# 6. Heatmap: numeric correlations
corr_cols = ["release_year", "year_added", "duration_number"]
```

Appendix A — Source Code (continued)

```
plt.figure(figsize=(6, 4))
sns.heatmap(
    df[corr_cols].corr(numeric_only=True),
    annot=True, cmap="coolwarm", vmin=-1, vmax=1
)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# 5. SAVE CLEANED DATA AND SUMMARY OUTPUTS
# ------------------------------------------------------------
df.to_csv("netflix_titles_cleaned.csv", index=False)
print("\nSaved cleaned dataset as: netflix_titles_cleaned.csv")

summary = pd.DataFrame({
    "Content Type": type_counts.index,
    "Count": type_counts.values
})
summary.to_csv("netflix_type_summary.csv", index=False)
print("Saved content-type summary as: netflix_type_summary.csv")

print("\nPROJECT COMPLETED.")
print("Use the real values printed above in your report; do not invent findings.")
print("Note: This project describes the dataset and does not prove why Netflix made a decision.")
```